

DataAgent — 系统架构

概述

DataAgent 是企业级智能数据分析平台，采用**前后端分离的 B/S 架构**。后端使用 Go 语言单二进制部署，Worker 和 Scheduler 作为同进程 goroutine 运行，异步任务通过 Redis Stream 协调。

系统提供两种交互模式：

- **Chat 模式**: 实时对话式数据查询，SSE 流式响应
- **Agent 模式**: 批量分析任务，支持同步执行和异步队列处理

Agent Service 作为所有用户请求的中央入口，管理 Session、编排 ADK Agent Engine 进行 LLM 推理和工具调用，并与 Skills（SQL 执行器、统计引擎、知识库搜索等）协调。

系统架构图

```
Client → API Gateway (JWT + RBAC + Rate Limit + Security Filter)
      → Agent Service (Session + ADK Agent Engine)
          → [LLM Router → Skills (SQL/Stats/KB/Email/...)]
      → Security Filter (Output) → Audit Log → Response
```

技术栈

层级	技术	版本
后端语言	Go	1.22+
Agent 框架	google.golang.org/adk	latest
业务数据库	MongoDB	7.0+
向量数据库	Milvus	2.4+
对象存储	SeaweedFS	RELEASE.2024+
缓存与消息队列	Redis	7.2+
密钥管理	HashiCorp Vault	1.18+
记忆系统	Mem0	latest
前端	React / Next.js	18 / 14
飞书 SDK	go-lark/lark	latest

服务一览

服务	描述
agent-service	中央协调器：Chat 同步、Agent 同步/异步、Session 管理
agent-engine	基于 ADK 的 Agent 引擎：LLM 路由、工具调用、安全审计、上下文窗口管理
worker-pool	Goroutine 池，消费 Redis Stream 中的异步任务
scheduler	基于 Cron 的定时任务触发器（写入 Redis Stream）+ Stats 统计收集
admin-service	管理后台 API（用户、角色、模型、任务、审计、Dashboard ROI）
im-module	飞书机器人集成（集成在主二进制，internal/service/im/，Webhook + 消息路由）
hermes-service	独立转发层，用于自由探索模式

数据存储

存储	用途
MongoDB	全部业务实体（用户、会话、任务、报告、产物、审计日志）
Milvus	知识库语义搜索的向量嵌入
SeaweedFS	二进制文件（会话工作区、产物文件内容）
Redis	查询缓存、消息队列（Stream）、会话数据
Mem0	多粒度记忆管理
Vault	密钥加密（API Key、数据库密码、JWT Secret）

关键设计决策

- 1. **单二进制部署**: 降低 MVP 运维复杂度。Worker 和 Scheduler 作为 goroutine 运行，非独立服务。
- 2. **Agent Service 作为唯一入口**: Chat 和 Agent 模式共享同一引擎，避免代码重复。
- 3. **Redis Stream 做异步队列**: 轻量级消息队列，无需额外中间件（不引入 RabbitMQ/Kafka）。
- 4. **LLM 智能分片**: 使用主 LLM 判断语义段落边界，不引入专用 embedding 模型。
- 5. **SQL AST 安全校验**: 通过 `pingcap/tidb/parser` 在语法层面拦截写入操作，不依赖 LLM prompt 约束。
- 6. **上下文窗口管理**: tiktoken-go (MIT) token 计数 + LLM 摘要压缩，保留最近 4 轮原文，旧消息压缩为摘要（~100 tokens/次，成本 ¥0.001）。不引入 Python 服务或专用压缩模型。
- 7. **Markdown AST 报告校验**: 通过 Markdown AST 解析提取标题层级校验章节完整性，替代正则匹配方式。
- 8. **Redis Stats 统计**: Scheduler 定时直接写入 Redis（AOF+RDB 持久化），记录 Agent/模型/Session/Task/Token 指标，Dashboard 实时聚合计算 ROI。
- 9. **MongoDB TTL 自动清理**: 日志类集合（审计日志/请求日志/通知/Token消耗）使用 TTL 索引自动过期，无需手动清理 Worker。
- 10. **IM 模块集成部署（仅 Chat 模式）**: 飞书 Bot 集成在主二进制（`internal/service/im/`），仅接入轻量办公 Chat API，不接入 Agent 批量任务和 Hermes 探索模式。

目录结构

```
data-agent/  
├── cmd/server/main.go      # 入口（单二进制）
```

```

├─ internal/
|   ├─ api/                # HTTP 层 (handler、middleware、router)
|   ├─ logic/              # 共用业务逻辑 (Skill + Handler + Service)
|   |   ├─ sql/            # SQL 生成 + AST 安全校验
|   |   ├─ stats/          # 统计分析 (回归/PCA/聚类)
|   |   ├─ knowledge/      # 混合搜索 + RRF 排序
|   |   └─ report/         # Markdown AST 报告校验
|   |   └─ openapi/        # OpenAPI 3.0 解析器
|   └─ service/            # 业务服务 (chat/agent/admin/im)
|   └─ domain/             # 领域模型、Agent 引擎、Skill 注册、安全审计
|   └─ worker/             # 异步任务 Worker Pool
|   └─ queue/              # Redis Stream 任务队列 + 死信队列
|   └─ scheduler/          # Cron 调度器
|   └─ infra/              # 基础设施 (MongoDB/Milvus/Redis/SeaweedFS)
|   └─ config/             # 配置管理
├─ skills/                 # Skill 实现
├─ frontend/               # React/Next.js 前端
├─ tests/                  # 测试
|   └─ ui/                  # Playwright E2E 测试 (MVP 占位)
├─ configs/                # YAML 配置文件
├─ docs/                   # 公开文档
├─ .agent/                 # AI Agent 指令 (SSOT)
├─ docker-compose.yml
└─ Makefile

```

参考文档

- [产品需求文档 \(PRD\)](#)
- [技术方案 RFC](#)
- [开发路线图](#)
- [.agent/memory/ARCHITECTURE.md](#) — AI Agent 用的详细架构文档